

EXAMEN 3ª EVALUACIÓN – DESENVOLVEMENTO WEB

CONTORNO SERVIDOR

Hora de inicio: 16:00

Hora de entrega: 18:15 – 18:20

1. CONTEXTO Y PROYECTO BASE

“Hospital” es una aplicación web de gestión hospitalaria desarrollada en PHP que simula el sistema de información utilizado en un centro médico para administrar los aparatos electrónicos instalados en sus distintas plantas.

La aplicación permite registrar y consultar información de aparatos médicos de tres tipos diferentes: desfibriladores, máquinas de rayos X y respiradores. Cada aparato almacena datos generales (nombre, marca, modelo, número de serie, planta, habitación, fecha de adquisición, estado) y características específicas según su tipo.

La aplicación cuenta con:

- Arquitectura MVC básica (Modelos, Vistas, Controladores)
- Conexión a base de datos mediante PDO
- Una clase abstracta Aparato y clases hijas: Desfibrilador, MaquinaRayosX, Respirador
- Base de datos única (*hospital_examen*) con tabla única (*aparatos*)
- Vistas: index.php, detalle.php, editar.php, eliminar.php, login.php, logout.php
- Además, se desarrollará una API REST con el framework Flight PHP

NOTA IMPORTANTE: En el código proporcionado encontraréis comentarios --- *EXAMEN* --- que indican el lugar exacto donde debéis realizar cada implementación. Seguid las pautas que aparecen en dichos comentarios.

Estructura de archivos proporcionada:

```
/hospital/
├── config/Database.php           (Conexión PDO + creación de tabla)
├── models/
│   ├── Aparato.php              (Clase abstracta base)
│   ├── Desfibrilador.php        (Hereda de Aparato)
│   ├── MaquinaRayosX.php        (Hereda de Aparato)
│   ├── Respirador.php           (Hereda de Aparato)
│   ├── Usuario.php              (Ya implementado)
│   ├── Auth.php                 (Gestiona la lógica del login)
│   └── AparatoRepository.php     (CRUD)
├── views/                       (Header y footer)
├── index.php                    (página principal)
├── detalle.php                  (vista para ver detalles de unaparato)
├── editar.php                   (formulario para editar aparatos)
├── eliminar.php                 (eliminación de aparatos)
├── login.php                    (formulario de login)
├── logout.php                   (cierre de sesión)
/API/
(archivos de configuración de Flight)
└── index.php                    (API REST con Flight PHP)
```

2. TAREAS A IMPLEMENTAR

TAREA 1: SISTEMA DE AUTENTICACIÓN, ROLES Y SEGURIDAD (2,5 puntos)

La aplicación consta inicialmente de un único usuario administrador (*david / maquinas*) que puede editar aparatos y acceder a un formulario de alta (mockup). El alumno deberá implementar lo siguiente:

- En `models/Auth.php`: añadir dos nuevos usuarios hardcodeados: *carlos / normal* con rol 'usuario', e *nicolas / arreglos* con rol 'tecnico'.
- En `editar.php`: proteger el acceso para que únicamente el administrador pueda entrar.
- En `eliminar.php`: proteger el acceso para que únicamente el técnico pueda eliminar aparatos.

- En index.php: ocultar los botones *Editar* y *Eliminar* del listado general cuando el usuario logueado no sea administrador.
- En index.php: mostrar el cuadro de aparatos obsoletos únicamente al técnico (ver más en la siguiente sección).
- En views/header.php: mostrar el nombre y el rol del usuario actual, así como un enlace para cerrar sesión.

Resumen de permisos:

Rol	Ver listado	Ver detalle	Ver obsoletos	Editar	Eliminar
admin	Sí	Sí	Sí	Sí	No
tecnico	Sí	Sí	Sí	No	Sí (solo desde obsoletos)
usuario	Sí	No	No	No	No

TAREA 2: FILTRO POR PLANTA, CUADRO DE SUSTITUCIÓN Y BOTÓN DE ELIMINAR (5 puntos)

El alumno deberá implementar un nuevo filtro por planta y un sistema que detecte automáticamente qué aparatos deben ser sustituidos, incluyendo un botón para eliminar dicho aparato de la base de datos.

- En models/AparatoRepository.php: completar el método *findAll()* para que admita un filtro por planta (0, 1 o 2).
- En index.php: añadir un desplegable junto al filtro de tipo para seleccionar la planta. Debe conservar la selección tras enviar el formulario.
- En models/Aparato.php: declarar el método abstracto *necesitaSustitucion()*. El método *getAntiguedad()* ya está implementado; no debe modificarse.

- En `models/Desfibrilador.php`: implementar `necesitaSustitucion()`. Un desfibrilador debe sustituirse si tiene más de 20 años y está en la planta 2.
- En `models/MaquinaRayosX.php`: implementar `necesitaSustitucion()`. Una máquina de rayos X debe sustituirse si su número de serie es menor a 10000.
- En `models/Respirador.php`: implementar `necesitaSustitucion()`. Un respirador debe sustituirse si tiene más de 15 años.
- En `index.php`: implementar un cuadro de alerta (solo visible para *tecnico*) que liste los aparatos obsoletos, mostrando nombre, tipo, ubicación, motivo de sustitución y un botón *Eliminar*.
- En `index.php`: cada aparato obsoleto del cuadro de alerta debe incluir un botón "Eliminar" que redirija a `eliminar.php?id=ID_DEL_APARATO`.
- En `models/AparatoRepository.php`: implementar el método `delete($id)` para ejecutar la sentencia SQL que elimine el aparato cuyo identificador coincida con el parámetro recibido.
- Implementar completamente `eliminar.php` que gestiona el formulario del botón y llama al método `delete($id)`.

TAREA 3: API REST CON FRAMEWORK FLIGHT PHP (2.5 puntos)

Desarrollar dos endpoints en el archivo `api.php` utilizando el framework Flight PHP. Se proporcionan datos de ejemplo y rutas guía ya implementadas.

- GET `/aparatos/antiguedad/@anios` — Devolver los aparatos cuya antigüedad sea menor al número de años indicado. Utilizar la función `calcularAntiguedad()` ya programada.
- POST `/aparatos/@id/estado` — Actualizar el estado de un aparato según la siguiente regla:
 - Mantenimiento → cambiar a Operativo
 - Fuera de servicio → cambiar a Mantenimiento
 - Operativo → no realizar cambio, devolver mensaje informativo